Monte Carlo Methods for Estimation

Mark M. Fredrickson (mfredric@umich.edu)

Module 1: Integrate

Computational Methods in Statistics and Data Science (DATASCI 406)

A New Thing to Estimate

Where we are

The first two videos estimated $\mathrm{E}\,(h(X))$ — a number we could in principle have got by calculus, if the calculus were tractable. The recipe never changed: **sample, apply h, average**.

Today with consider

> *We have data $X_1, \ldots, X_n$ and a rule $\hat{\theta}(X_1, \ldots, X_n)$ for guessing θ. Is it any good? Is it better than the other rule?*

Now we want to know about $\mathrm{E}\left(h(\hat{\theta})\right)$ – which is an **operating characteristic** of the estimator $\hat{\theta}$. To draw from the distribution of $\hat{\theta}$, we will draw **random vectors** $X_1, \ldots, X_n$.

Estimators are random variables

An **estimator** is a function of the sample, so it is a random variable. Its distribution is the **sampling distribution** of $\hat{\theta}$.

Everything we might want to know about the estimator is a feature of that distribution:

- Is it centered on θ? (**unbiased**)
- How precise of an estimator is it? (**variance**)
- How far from θ is it, typically? (**mean squared error**)

Each is an **expectation** over the sampling distribution. So each is an integral, and each is a job for Monte Carlo.

The simulation loop

Only one thing changes from the last two videos: **what one draw is**. Before it was a number X_i. Now it is an **entire dataset**.

```
theta_hats <- replicate(k, {
  x <- rmodel(n)          # simulate a whole dataset of size n
  estimator(x)            # collapse it to one number
})
```

`theta_hats` holds k draws from the sampling distribution. From there it is ordinary Monte Carlo: to estimate $\mathrm{E}\left(g(\hat{\theta})\right)$, average g over those draws.

Do not confuse n and k

	n	k
what it is	size of one dataset	number of replications
set by	the problem	your patience
bigger means	the **estimator** does better	**you** know it better
shrinks	$\text{Var}\left(\hat{\theta}\right)$	the error in our *estimate* of $\text{Var}\left(\hat{\theta}\right)$

Raising k does **not** make the estimator better. It sharpens our description of it.

So if a simulation reports an MSE of 0.04, that 0.04 is itself a Monte Carlo estimate, and it deserves a confidence interval like any other.

A concrete question

$X_1, \ldots, X_n \overset{\text{iid}}{\sim} U(0, \theta)$, with θ unknown. Two familiar estimators:

- **Method of moments**: match population moments to sample moments.
- **Maximum likelihood**: pick the θ making the data most probable.

Two respectable principles, giving two different numbers. This is exactly the Model U situation from your project, so let us do both properly.

Method of moments

The population mean is a function of θ. Work it out:

$$\mathrm{E}(X) = \int_0^\theta x \cdot \frac{1}{\theta}\, dx = \frac{1}{\theta}\left[\frac{x^2}{2}\right]_0^\theta = \frac{\theta}{2}$$

Invert it — $\theta = 2\,\mathrm{E}(X)$ — and replace the population mean by the sample mean:

$$\tilde{\theta} = 2\bar{X}$$

It is **unbiased**, and the proof is that inversion run forwards:

$$\mathrm{E}\left(\tilde{\theta}\right) = 2\,\mathrm{E}\left(\bar{X}\right) = 2 \cdot \frac{\theta}{2} = \theta$$

That worked because θ is a **linear** function of $\mathrm{E}(X)$ here. Method-of-moments estimators are not unbiased in general.

Maximum likelihood

The likelihood is the joint density, read as a function of θ:

$$L(\theta) = \prod_{i=1}^{n} \frac{1}{\theta} \mathbf{1}(0 \le x_i \le \theta) = \theta^{-n} \mathbf{1}\Big(\theta \ge \max_i x_i\Big)$$

because all n indicators are 1 exactly when θ clears the largest observation.

Two observations finish it:

- if $\theta < \max_i x_i$ then $L(\theta) = 0$;
- if $\theta \ge \max_i x_i$ then $L(\theta) = \theta^{-n}$, **strictly decreasing**.

So L is largest at the smallest feasible θ: $\hat{\theta} = \max_i X_i$.

What we did not do

We never set a derivative to zero.

The support of f **depends on the parameter**, so L jumps from 0 up to θ^{-n} at $\theta = \max_i x_i$. It is discontinuous exactly at its maximizer, and $\partial_\theta \log L = 0$ has no solution anywhere.

This is the standard cautionary example. Differentiate-and-solve is a **method for finding interior maxima of smooth likelihoods**, not the definition of maximum likelihood. The definition is "the θ that maximizes L", and here you get it by reading the function.

Worth keeping: in your project the Model B likelihood **is** smooth in its parameter, and there the derivative does the work.

Bias, Variance, and MSE

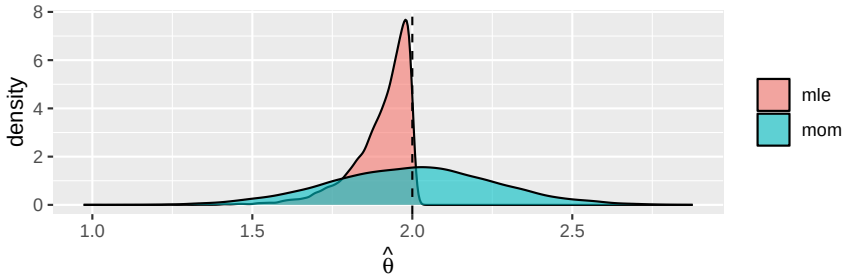
Simulating both sampling distributions

Take $\theta = 2$ and $n = 20$.

```
k <- 10000; n <- 20; theta <- 2
datasets <- replicate(k, runif(n, 0, theta), simplify = FALSE)
mle <- map_dbl(datasets, max)
mom <- map_dbl(datasets, ~ 2 * mean(.x))
```

Both estimators are applied to the **same** simulated datasets — the *common random numbers* idea from the last video. Any difference we see is then the estimators differing, not the draws differing.

The two sampling distributions



Not even the same *shape*. The MLE is a maximum of numbers below θ, so it is **always too small**. The method of moments is a sample mean, so the CLT makes it symmetric.

Bias

The **error** is $\hat{\theta} - \theta$; the **bias** is the average error:

$$\text{Bias}(\hat{\theta}) = \text{E}\left(\hat{\theta} - \theta\right) = \text{E}\left(\hat{\theta}\right) - \theta$$

An estimator is **unbiased** when this is zero.

An expectation, so we estimate it by averaging. And because it is a sample mean over the k replications, `t.test` reports its Monte Carlo error for free.

```
as.numeric(t.test(mle - theta)$conf.int)
```

```
## [1] -0.09887 -0.09524
```

```
as.numeric(t.test(mom - theta)$conf.int)
```

```
## [1] -0.006655  0.003347
```

Unbiased is not the same as good

```
c(var(mle), var(mom))
```

```
## [1] 0.008559 0.065088
```

The **biased** estimator has about **one eighth** the variance of the unbiased one.

So we cannot rank them on bias alone, and we cannot rank them on variance alone. We need one number that charges an estimator for both.

Mean squared error

$$\text{MSE}(\hat{\theta}) = \text{E}\left((\hat{\theta} - \theta)^2\right)$$

Add and subtract $\text{E}\left(\hat{\theta}\right)$ inside the square:

$$\begin{aligned}\text{E}\left((\hat{\theta} - \theta)^2\right) &= \text{E}\left(\left(\left[\hat{\theta} - \text{E}\left(\hat{\theta}\right)\right] + \left[\text{E}\left(\hat{\theta}\right) - \theta\right]\right)^2\right) \\ &= \text{Var}\left(\hat{\theta}\right) + \text{Bias}(\hat{\theta})^2 + 2\,\text{Bias}(\hat{\theta})\,\text{E}\left(\hat{\theta} - \text{E}\left(\hat{\theta}\right)\right)\end{aligned}$$

The last term is zero, because $\text{E}\left(\hat{\theta} - \text{E}\left(\hat{\theta}\right)\right) = 0$. So

$$\textbf{MSE} = \textbf{variance} + \textbf{bias}^2$$

For unbiased estimators, minimizing MSE *is* minimizing variance — but that is a special case, not the general rule.

Estimating MSE

One more expectation, so one more sample mean — this time of the squared errors.

```
as.numeric(t.test((mle - theta)^2)$conf.int)
```

```
## [1] 0.01727 0.01869
```

```
as.numeric(t.test((mom - theta)^2)$conf.int)
```

```
## [1] 0.06330 0.06686
```

The **biased** estimator wins by a factor of more than three. Its small variance more than pays for its bias.

Checking the decomposition

```
c(var(mle) + mean(mle - theta)^2, mean((mle - theta)^2))
```

```
## [1] 0.01798 0.01798
```

The two agree to the printed precision, as they must.

A good habit: whenever a simulation reports bias, variance and MSE, check that they add up.

What the algebra predicts

Both estimators can be worked out exactly, which gives us something to check the simulation against. For $\tilde{\theta} = 2\bar{X}$:

$$\text{Bias} = 0, \qquad \operatorname{Var}\left(\tilde{\theta}\right) = \frac{4\operatorname{Var}(X)}{n} = \frac{\theta^2}{3n}$$

For $\hat{\theta} = \max_i X_i$ the CDF is $P(\hat{\theta} \le t) = (t/\theta)^n$, giving density nt^{n-1}/θ^n and

$$\mathrm{E}\left(\hat{\theta}\right) = \frac{n}{n+1}\theta \implies \text{Bias} = -\frac{\theta}{n+1}, \qquad \text{MSE} = \frac{2\theta^2}{(n+1)(n+2)}$$

The MLE's bias shrinks like $1/n$ and its MSE like $1/n^2$ — the $O(n^{-2})$ we are about to see in the table.

Checking the simulation against the algebra

	exact	lo	hi	covers
mle bias	-0.095238	-0.098871	-0.095244	FALSE
mom bias	0.000000	-0.006655	0.003347	TRUE
mle mse	0.017316	0.017271	0.018685	TRUE
mom mse	0.066667	0.063304	0.066863	TRUE

Three of the four intervals cover the exact value; the fourth **misses by six millionths**. That is not a bug — a 95% interval is meant to miss one time in twenty, and we ran four.

Check your simulation where you *can* do the algebra. It is the only place you can check it at all.

Does the ranking survive a change in n?

```
compare <- function(n, k = 10000, theta = 2) {
  d <- replicate(k, runif(n, 0, theta), simplify = FALSE)
  est <- list(mle = map_dbl(d, max),
              mom = map_dbl(d, ~ 2 * mean(.x)))
  map_dfr(est, ~ tibble(bias = mean(.x - theta), var = var(.x),
                        mse = mean((.x - theta)^2)),
          .id = "estimator") %>%
    mutate(n = n, .before = 1)
}
```

Three sample sizes

n	estimator	bias	var	mse
5	mle	-0.3355	0.0805	0.1931
5	mom	-0.0056	0.2658	0.2658
20	mle	-0.0924	0.0079	0.0165
20	mom	0.0053	0.0672	0.0673
80	mle	-0.0246	0.0006	0.0012
80	mom	-0.0004	0.0168	0.0168

Each step **quadruples** n.

Reading the table

- Both MSEs fall as n grows: more data helps both rules.
- They do not fall at the **same rate**. Quadrupling n divides the method-of-moments MSE by about 4 — the $O(n^{-1})$ rate of any sample mean. It divides the MLE's by 12 to 14, heading for 16, because that MSE is $O(n^{-2})$.
- So the MLE does not merely win — **its lead widens with** n.

Comparing two estimators at one sample size tells you about that sample size. Sweeping n tells you which one you want as data gets cheaper.

When the Machinery Lies

A harder problem

$X_1, \ldots, X_n \overset{\text{iid}}{\sim} \text{Cauchy}(\theta)$, with density

$$f(x) = \frac{1}{\pi} \frac{1}{1 + (x - \theta)^2}$$

θ is the **median**. Two obvious rules for it:

- the **sample median**, which matches the definition;
- the **sample mean**, which is what we always reach for.

The Cauchy has **no mean**: $\int |x| f(x)\, dx$ diverges. But $\bar{X}$ is a perfectly well-defined number for any finite sample, so nothing stops us computing it. Let us see what the simulation says.

Simulating

```
k <- 10000; n <- 20; true_theta <- 10
cdata <- replicate(k, rcauchy(n, location = true_theta),
                   simplify = FALSE)
cmed <- map_dbl(cdata, median)
cmean <- map_dbl(cdata, mean)
```

```
range(cmed)
```

```
## [1]  8.289 11.755
```

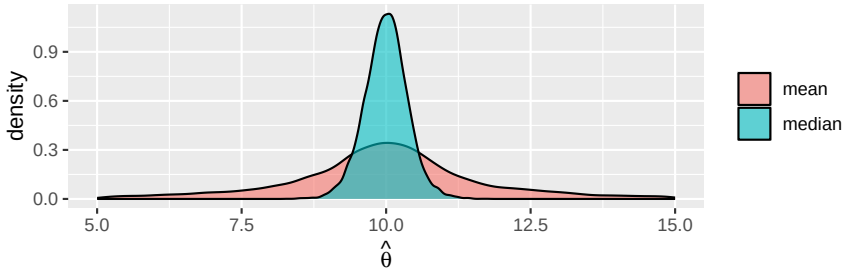
```
range(cmean)
```

```
## [1] -4241  8067
```

The median never strays far. The mean, over the same datasets, produced values thousands of units from a truth of 10.

The two sampling distributions

```
## Warning: Removed 1279 rows containing non-finite outside the scale
## range (`stat_density()`).
```



The mean's distribution has been **cropped** to fit the slide — that is the point of the picture.

Why

This is not bad luck. For $X_i \overset{\text{iid}}{\sim} \text{Cauchy}(\theta)$ it is a theorem that

$$\bar{X} \sim \text{Cauchy}(\theta) \quad \textbf{exactly, for every } n$$

Averaging n Cauchy draws hands you back a single Cauchy draw. **The sample mean of 10,000 observations is no better than one observation.** The usual intuition — averaging cancels noise — is a consequence of the CLT, and the CLT needs a finite variance.

What the machinery reports anyway

```
mean(cmean - true_theta)
```

```
## [1] 1.256
```

```
mean((cmean - true_theta)^2)
```

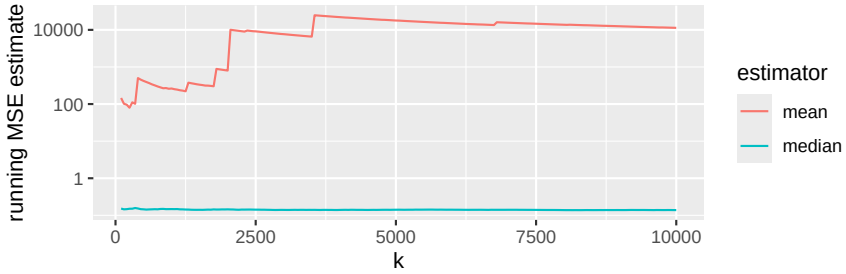
```
## [1] 11331
```

Two finite numbers, printed without complaint. Both are **meaningless**: $\bar{X}$ is Cauchy, so $\mathrm{E}\left(\bar{X}\right)$ does not exist and $\mathrm{E}\left((\bar{X}-\theta)^2\right) = \infty$.

Run it again with a different seed and the second number changes by orders of magnitude.

Monte Carlo will not warn you. It returns the average of what it sampled, and that average always exists.

The symptom you can see



The median's estimate settles. The mean's jumps every time a new extreme value arrives and never settles — the same picture as the infinite-variance estimator in session 1's exercises.

Introducing the trimmed mean

The median works because it ignores the tails. The **trimmed mean** interpolates: discard the most extreme observations at each end, average the rest.

```
ctrim <- map_dbl(cdata, mean, trim = 0.25)
map_dbl(list(median = cmed, trimmed = ctrim),
        ~ mean((.x - true_theta)^2))
```

```
##  median trimmed
##  0.1384  0.1592
```

Trimming a quarter from each end beats the median: it uses middle observations the median throws away, and still never lets an outlier in.

There is no row for the mean — not because it lost, but because **the quantity does not exist**.

Confidence Intervals and Coverage

A different kind of question

A point estimate answers "what is θ?" An interval answers "how sure are we?"

A $(1-\alpha)$ **confidence interval** is a pair of statistics (A, B), both computed from the data, with

$$P(A \leq \theta \leq B) = 1 - \alpha$$

The probability is over the **interval**, not over θ: θ is a fixed unknown number, and the endpoints are what move from sample to sample.

CI for sample mean

Recall the **central limit theorem** states $(\bar{X}_n - \mathrm{E}(X))/(\sigma/\sqrt{n})$ is approximately $N(0,1)$ in large samples so that

$$P\left(-z_{\alpha/2} \leq \frac{\bar{X}_n - \mathrm{E}(X)}{\sigma/\sqrt{n}} \leq z_{\alpha/2}\right) \approx 1-\alpha$$

where $z_{\alpha/2}$ is the upper $\alpha/2$ quantile of $N(0,1)$, or `qnorm(1 - alpha/2)`. For $\alpha = 0.05$ that is the familiar 1.96.

Read what this says: a **ratio** we cannot compute (it contains $\mathrm{E}(X)$) falls between two numbers we can, with probability $1-\alpha$.

Now rearrange the event until it is a statement about $\mathrm{E}(X)$ instead.

Rearranging to surround $\mathsf{E}(X)$

Multiply through by $\sigma/\sqrt{n}$, subtract $\bar{X}_n$, then multiply by -1 — which reverses both inequalities:

$$-z_{\alpha/2} \le \frac{\bar{X}_n - \mathsf{E}(X)}{\sigma/\sqrt{n}} \le z_{\alpha/2} \iff -z_{\alpha/2}\frac{\sigma}{\sqrt{n}} \le \bar{X}_n - \mathsf{E}(X) \le z_{\alpha/2}\frac{\sigma}{\sqrt{n}}$$

$$\iff \bar{X}_n - z_{\alpha/2}\frac{\sigma}{\sqrt{n}} \le \mathsf{E}(X) \le \bar{X}_n + z_{\alpha/2}\frac{\sigma}{\sqrt{n}}$$

Every step is an **equivalence**, so the probability never changed:

$$P\Big(\underbrace{\bar{X}_n - z_{\alpha/2}\tfrac{\sigma}{\sqrt{n}}}_{A} \le \mathsf{E}(X) \le \underbrace{\bar{X}_n + z_{\alpha/2}\tfrac{\sigma}{\sqrt{n}}}_{B} \Big) \approx 1 - \alpha$$

Exactly the (A, B) from the previous slide. **Nothing moved but the algebra**: $\mathsf{E}(X)$ is still fixed, and the randomness is still $\bar{X}_n$ — now in *both* endpoints.

We do not know σ

A and B must be **statistics** — computable from the data — and σ is not one. Replace it by the sample standard deviation S.

This is not free. The ratio $(\bar{X}_n - \mathrm{E}(X))/(S/\sqrt{n})$ now has a random quantity in the denominator too, so it is no longer standard Normal: it has **heavier tails**, more so the smaller n is. For Normal data it is exactly t_{n-1}, so we widen the interval with t quantiles:

$$\bar{X}_n \pm t_{\alpha/2}(n-1)\,\frac{S}{\sqrt{n}}$$

This is the interval `t.test` reports.

For **non-Normal** data we now approximate twice: once in the CLT, once in calling that ratio a t. Whether it still delivers $1-\alpha$ is a question about a probability — so simulation can settle it.

Coverage is a probability, so it is an expectation

$$\text{coverage} = P(A \le \theta \le B) = \mathrm{E}\left(\mathbf{1}(A \le \theta \le B)\right)$$

```
covers <- function(x, truth) {
  ci <- t.test(x)$conf.int
  ci[1] <= truth && truth <= ci[2]
}
```

The one new wrinkle: **we have to know the truth** to check coverage. That is what makes this a simulation rather than a data analysis — we get to set the truth.

Coverage for skewed data

$X_i \overset{\text{iid}}{\sim} \text{Exp}(1)$, so the truth is $\text{E}(X) = 1$. The CLT states that as n gets large X_n should be approximately normal, which can be used to create a confidence interval. How well does it work?

```
k <- 10000
map_dbl(set_names(c(5, 10, 30, 100)),
        ~ mean(replicate(k, covers(rexp(.x), truth = 1))))
```

```
##      5     10     30    100
## 0.8858 0.9010 0.9297 0.9420
```

The nominal 95% interval does not deliver 95% at small n, and the shortfall closes only slowly. An interval that promises 95% and delivers 90% is **anti-conservative**: wrong twice as often as advertised.

Put an interval on the coverage

The coverage estimate is a proportion out of k independent trials, so it is Binomial and `binom.test` gives an exact interval.

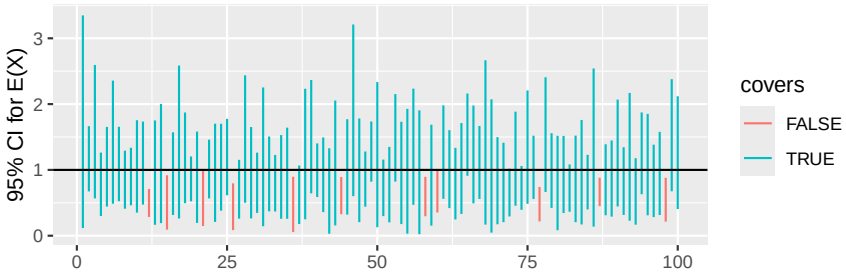
```
hits <- sum(replicate(k, covers(rexp(10), truth = 1)))
as.numeric(binom.test(hits, k, conf.level = 0.99)$conf.int)
```

```
## [1] 0.8918 0.9074
```

Entirely below 0.95, so the under-coverage at $n = 10$ is real and not simulation noise.

Prefer `binom.test` to `t.test` whenever the thing you averaged was an **indicator**: it is exact for any k, it cannot return an endpoint below 0 or above 1, and it does not lean on the CLT.

Seeing it



The truth is a fixed line; the **intervals** are what jump around. The misses are nearly all below it — a right-skewed sample usually understates its own spread.

Coverage is not the only thing

Coverage alone is easy to achieve: the interval $(-\infty, \infty)$ has 100% coverage and says nothing.

So report **width** alongside it.

```
mean(replicate(1000, diff(t.test(rexp(10))$conf.int)))
```

```
## [1] 1.311
```

One procedure beats another when it holds **at least the advertised coverage at a smaller expected width**. Either number alone is half an answer — the same lesson as bias and variance, in a new costume.

Summary

- An estimator is a random variable; its distribution is the **sampling distribution**. Simulate it by simulating whole **datasets**, each collapsed to one number.
- **Bias**, **variance**, **MSE** and **coverage** are all expectations over that distribution, so all four use the same sample-mean recipe as video 1.
- **MSE = variance + bias**^{2}, so a biased estimator can win — for $U(0, \theta)$ it wins easily. Check MSE, and sweep n, since the ranking can depend on both.
- Your simulation output is itself an estimate. Put a confidence interval on it: `t.test` for an average, `binom.test` for a proportion.
- Run competing estimators on the **same** simulated datasets.
- Monte Carlo prints a number whether or not the quantity exists. With heavy tails the number is noise — watch the running estimate.